SCRIPT-CONTROLLED ACL – RESTRICT RECORD ACCESS BASED ON FIELD VALUE

PHASE 1: BRAINSTORMING & IDEATION PHASE

Introduction

The project focuses on implementing **Script-Controlled Access Control Lists (ACLs)** in the ServiceNow platform to provide secure and controlled access to records. ACLs help determine whether users are authorized to view, access, or modify specific records based on defined rules and conditions.

In this project, a script-based ACL is configured to control record access according to specific field values. The implementation involves analyzing the requirements, designing the access-control logic, configuring the ACL, and testing its functionality.

The main purpose of the project is to improve **data security, privacy, and controlled access** within the ServiceNow application. The complete implementation process is presented through screenshots and step-by-step documentation.

Record-level security is one of the most important parts of any enterprise application. In a platform like ServiceNow, a single table can hold thousands of records that belong to different departments, branches or customers. If every user with basic table access could see every record, sensitive information could easily be exposed to people who have no need to see it. Access Control Lists solve this problem by evaluating, at the moment a user tries to read, create, write or delete a record, whether that user is permitted to perform the operation.

A standard ACL uses roles and simple conditions. A **script-controlled ACL** goes one step further: it contains a small piece of server-side JavaScript that runs each time the rule is evaluated and returns *true* (access granted) or *false* (access denied). This makes it possible to combine roles, field values and other logic in a way that a plain role check cannot express.

In this project a custom table called **Institution Details** was created to store student and faculty information such as roll number, student name, faculty name, branch, email, phone number and description. The goal is to make sure that records of a particular branch (EEE) are visible only to the users who are authorized for that branch, while administrators keep full control and every other user is blocked.

The Brainstorming and Ideation Phase focused on identifying the need for secure record access in ServiceNow. The idea of using an Access Control List (ACL) was developed to control user access based on specific conditions. A script-controlled ACL approach was planned to provide flexible and secure access control.

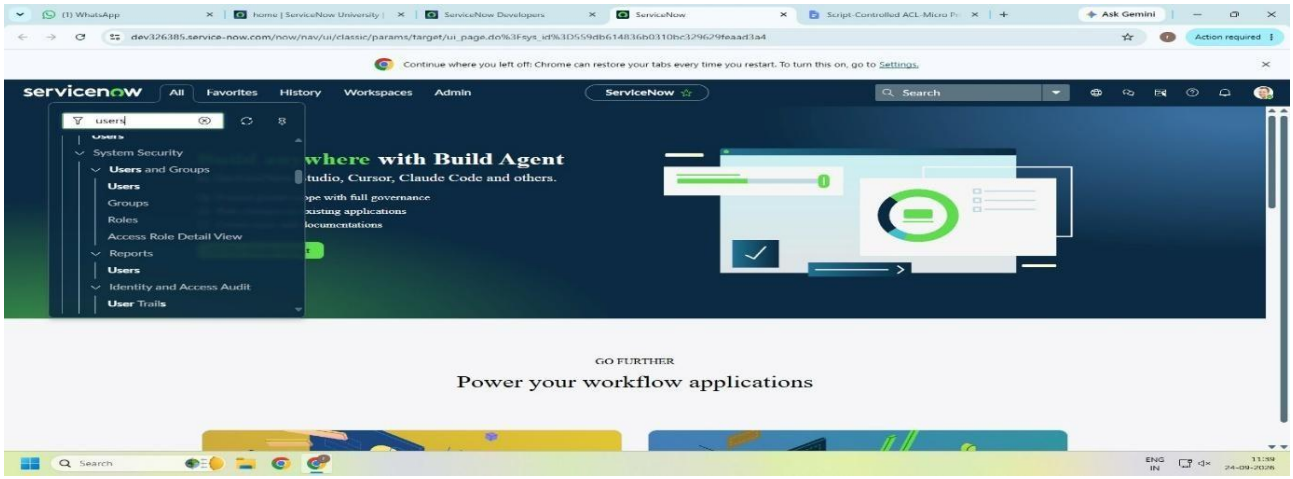
In this phase the team studied why access to records has to be restricted and which ServiceNow feature is best suited to do it. The discussion moved from a general security concern, that is, anybody can see everything, to a concrete solution: an ACL whose decision is made by a script and whose scope is limited by a field value on the record. The steps below show how the idea was formed, from the requirement to the choice of ACL and finally to the plan for scripting.

1.1 Identifying the Access Control Requirement

The requirement for controlling user access to specific ServiceNow records was identified. The system should allow authorized users to access the required records while preventing unauthorized access.

The first task was to state clearly what the system must do. Two requirements were noted. First, users who are authorized must be able to open and work with the records that belong to them. Second, users who are not authorized must not be able to see or change those records, even if they can log in to the instance and navigate to the same table.

The screenshot shows the ServiceNow instance home page, where the application navigator is used to search for the modules that will be needed during the project, starting with **Users**. This is the starting point for creating the test users and roles that will later be used to prove that the restriction works.

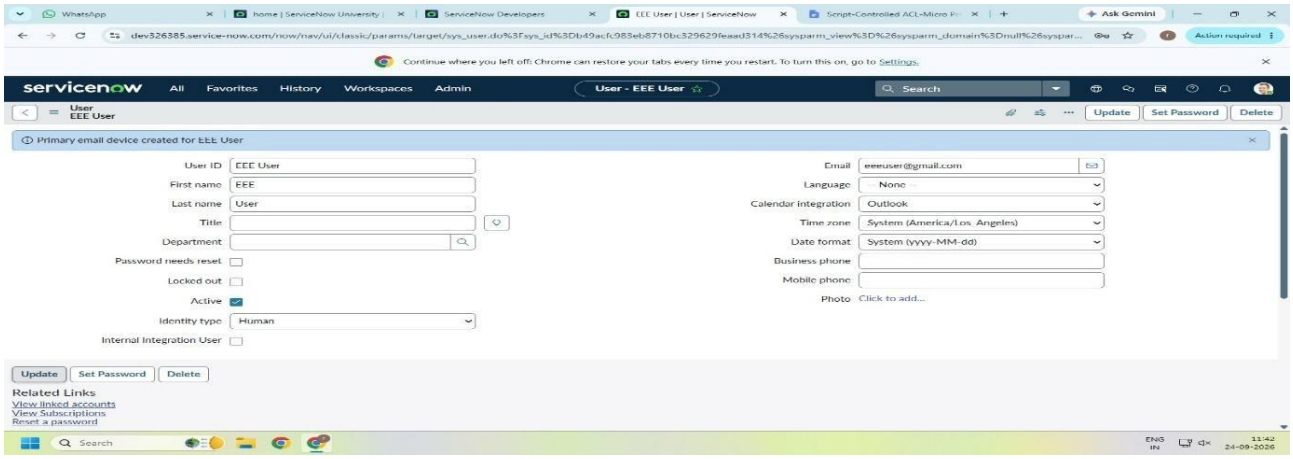


1.2 Understanding Record-Level Security

The need for record-level security was analyzed to protect sensitive information. Access to individual records should be controlled according to the defined access requirements.

Table-level security only decides whether a user can open a table at all. Record-level security is finer: it decides which individual rows of that table a user may see. In the Institution Details table, for example, every row carries a branch value. A student or faculty member of one branch should not be able to browse the records of another branch, so the security decision has to be taken row by row.

The screenshot shows the user record of a test user named **EEE User**. This user is active, has a first name and last name, and will be used later to log in (or be impersonated) and check what an EEE-branch user is allowed to see.

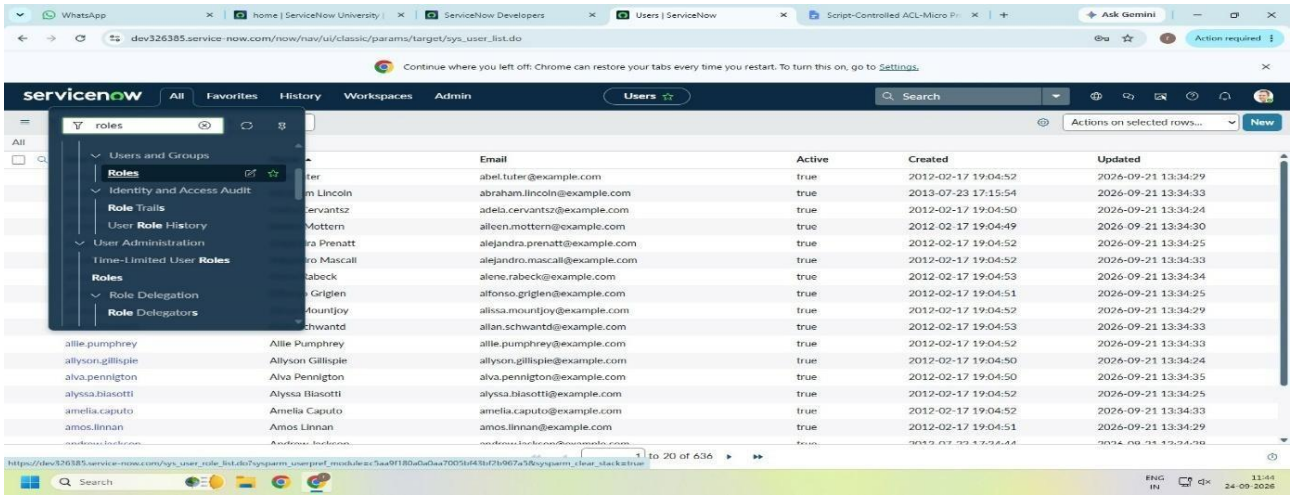


1.3 Selecting ACL-Based Access Control

ServiceNow Access Control Lists (ACLs) were selected as the method for managing record access. ACLs help determine whether users are permitted to access records based on specified rules and conditions.

ServiceNow offers several ways to protect data, such as roles, business rules, data policies and ACLs. ACLs were chosen because they are enforced by the platform itself on every route of access, whether the user opens a list, a form, a report or calls a web service. This means the rule cannot be bypassed simply by changing the way the data is requested.

Every ACL is made of a type, an operation (read, write, create, delete), the table it protects, and the conditions that must be satisfied: required roles, a data condition on the record fields and an optional script. The screenshot shows the Roles module opened from the navigator, because roles are the first ingredient of the ACL: the custom role will identify who is entitled to the restricted records.



1.4 Planning Script-Controlled Access

A script-controlled ACL approach was planned to provide flexible access control based on specific field values and conditions. The script logic would determine whether access should be granted or restricted.

A plain condition can say “the role must be X” or “the branch must be Y”. A script can express a complete decision: administrators always pass, users holding the branch role pass, and everybody else fails. Because the script returns a Boolean, the logic is easy to read and easy to change later.

The screenshot shows the creation of a new role record. The role is given a name and saved with the Submit button. This role is later assigned to the users who should have access, and the script checks for it with *gs.hasRole()*.

